One Pass Assembler

CS 348
Implementation of Programming Languages Lab
Computer Science and Engineering Department
Indian Institute of Technology
Guwahati

Contents

One Pass Assembler Overview

Why One Pass Assembler?

- Faster assembly
- Suitable for small programs
- Used in early systems and embedded environments
- Avoids storing full intermediate code

Core Concept

- Program is scanned only once
- Symbol resolution done immediately
- Forward references handled using fix-up lists
- Machine code generated on the fly

Forward Reference Challenge

A **forward reference** occurs when a symbol is used before it is defined.

In One Pass Assembler,

- Only one scan of source program is allowed.
- Machine code must be generated immediately.
- No second pass is available to fix unresolved addresses.

How to generate correct address field if symbol is undefined?

Forward Reference Challenge

```
MOVER AREG, 103
JMP LOOP
ADD BREG, 102
LOOP ADD AREG, BREG
```

In the above code LOOP is encountered before it is defined.

Backpatching

Backpatching is the process of filling in missing addresses of forward-referenced symbols after their definition is encountered.

Steps:

- Insert temporary/dummy address.
- Store the location where correction is needed.
- When symbol is defined , update all stored locations with correct address.

Backpatching Implementation

Usually done in two ways, Table of incomplete Instructions or Forward Reference list.

Location	Incomplete Instruction/Symbol
100	A
105	LOOP
106	ONE

Sym_no	Symbol	Address	Defined	Forward Reference
1	A	-	No	100
2	LOOP	107	Yes	105
3	ONE	-	No	106

Example

Source Program

```
START 100
MOVER AREG, A
PRINT B
ADD BREG, ='9'
SUB BREG, D
COMP CREG, ='23'
LTORG

A DC 3
LABEL: EQU A
ORIGIN 500
L1: MULT CREG, ='7'
B DC 10
MOVEM CREG, ='7'
D DC 8
END
```

Target Code

LC No	Incomplete Instruction

Sym_no	Symbol	Address

Lit_no	Literal	Address

Example

Source Program

```
START 100
MOVER AREG, A
PRINT B
ADD BREG, ='9'
SUB BREG, D
COMP CREG, ='23'
LTORG

A DC 3
LABEL: EQU A
ORIGIN 500
L1: MULT CREG, ='7'
B DC 10
MOVEM CREG, ='7'
D DC 8
END
```

Target Code

```
01 -  100
```

LC No	Incomplete Instruction

Sym_no	Symbol	Address

Lit_no	Literal	Address

Example

Source Program

```
START 100
MOVER AREG, A
PRINT B
ADD BREG, ='9'
SUB BREG, D
COMP CREG, ='23'
LTORG

A DC 3
LABEL: EQU A
ORIGIN 500
L1: MULT CREG, ='7'
B DC 10
MOVEM CREG, ='7'
D DC 8
END
```

Target Code

```
    01 -  100
100)01 01 ---
```

LC No	Incomplete Instruction
100	A

Sym_no	Symbol	Address
1	A	

Lit_no	Literal	Address

Example

Source Program

```
START 100
MOVER AREG, A
PRINT B
ADD BREG, ='9'
SUB BREG, D
COMP CREG, ='23'
LTORG

A DC 3
LABEL: EQU A
ORIGIN 500
L1: MULT CREG, ='7'
B DC 10
MOVEM CREG, ='7'
D DC 8
END
```

Target Code

```
    01 -  100
100)01 01 ---
101)09 -  ---
```

LC No	Incomplete Instruction
100	A
101	B

Sym_no	Symbol	Address
1	A	
2	B	

Lit_no	Literal	Address

Example

Source Program

```
START 100
MOVER AREG, A
PRINT B
ADD BREG, ='9'
SUB BREG, D
COMP CREG, ='23'
LTORG

A DC 3
LABEL: EQU A
ORIGIN 500
L1: MULT CREG, ='7'
B DC 10
MOVEM CREG, ='7'
D DC 8
END
```

Target Code

```
    01 -  100
100)01 01 ---
101)09 -  ---
102)03 02 ---
```

LC No	Incomplete Instruction
100	A
101	B
102	='9'

Sym_no	Symbol	Address
1	A	
2	B	

Lit_no	Literal	Address
1	='9'	

Example

Source Program

```
START 100
MOVER AREG, A
PRINT B
ADD BREG, ='9'
SUB BREG, D
COMP CREG, ='23'
LTORG

A DC 3
LABEL: EQU A
ORIGIN 500
L1: MULT CREG, ='7'
B DC 10
MOVEM CREG, ='7'
D DC 8
END
```

Target Code

```
    01 -  100
100)01 01 ---
101)09 -  ---
102)03 02 ---
103)04 02 ---
```

LC No	Incomplete Instruction
100	A
101	B
102	='9'
103	D

Sym_no	Symbol	Address
1	A	
2	B	
3	D	

Lit_no	Literal	Address
1	='9'	

Example

Source Program

```
START 100
MOVER AREG, A
PRINT B
ADD BREG, ='9'
SUB BREG, D
COMP CREG, ='23'
LTORG

A DC 3
LABEL: EQU A
ORIGIN 500
L1: MULT CREG, ='7'
B DC 10
MOVEM CREG, ='7'
D DC 8
END
```

Target Code

```
    01 -  100
100)01 01 ---
101)09 -  ---
102)03 02 ---
103)04 02 ---
104)08 03 ---
```

LC No	Incomplete Instruction
100	A
101	B
102	='9'
103	D
104	='23'

Sym_no	Symbol	Address
1	A	
2	B	
3	D	

Lit_no	Literal	Address
1	='9'	
2	='23'	

Example

Source Program

```
START 100
MOVER AREG, A
PRINT B
ADD BREG, ='9'
SUB BREG, D
COMP CREG, ='23'
LTORG

A DC 3
LABEL: EQU A
ORIGIN 500
L1: MULT CREG, ='7'
B DC 10
MOVEM CREG, ='7'
D DC 8
END
```

Target Code

```
    01 -  100
100)01 01 ---
101)09 -  ---
102)03 02 ---
103)04 02 ---
104)08 03 ---
105)-  -  009
```

LC No	Incomplete Instruction
100	A
101	B
102	='9'
103	D
104	='23'

Sym_no	Symbol	Address
1	A	
2	B	
3	D	

Lit_no	Literal	Address
1	='9'	105
2	='23'	

Example

Source Program

```
START 100
MOVER AREG, A
PRINT B
ADD BREG, ='9'
SUB BREG, D
COMP CREG, ='23'
LTORG

A DC 3
LABEL: EQU A
ORIGIN 500
L1: MULT CREG, ='7'
B DC 10
MOVEM CREG, ='7'
D DC 8
END
```

Target Code

```
     01 -  100
100)01 01 ---
101)09 -  ---
102)03 02 ---
103)04 02 ---
104)08 03 ---
105)-  -  009
106)-  -  023
```

LC No	Incomplete Instruction
100	A
101	B
102	='9'
103	D
104	='23'

Sym_no	Symbol	Address
1	A	
2	B	
3	D	

Lit_no	Literal	Address
1	='9'	105
2	='23'	106

Example

Source Program

```
START 100
MOVER AREG, A
PRINT B
ADD BREG, ='9'
SUB BREG, D
COMP CREG, ='23'
LTORG

A DC 3
LABEL: EQU A
ORIGIN 500
L1: MULT CREG, ='7'
B DC 10
MOVEM CREG, ='7'
D DC 8
END
```

Target Code

```
    01 -  100
100)01 01 ---
101)09 -  ---
102)03 02 ---
103)04 02 ---
104)08 03 ---
105)-  -  009
106)-  -  023
107)-  -  003
```

LC No	Incomplete Instruction
100	A
101	B
102	='9'
103	D
104	='23'

Sym_no	Symbol	Address
1	A	107
2	B	
3	D	

Lit_no	Literal	Address
1	='9'	105
2	='23'	106

Example

Source Program

```
START 100
MOVER AREG, A
PRINT B
ADD BREG, ='9'
SUB BREG, D
COMP CREG, ='23'
LTORG

A DC 3
LABEL: EQU A
ORIGIN 500
L1: MULT CREG, ='7'
B DC 10
MOVEM CREG, ='7'
D DC 8
END
```

Target Code

```
    01 -  100
100)01 01 ---
101)09 -  ---
102)03 02 ---
103)04 02 ---
104)08 03 ---
105)-  -  009
106)-  -  023
107)-  -  003
No code
No code
```

LC No	Incomplete Instruction
100	A
101	B
102	='9'
103	D
104	='23'

Sym_no	Symbol	Address
1	A	107
2	B	
3	D	
4	LABEL	107

Lit_no	Literal	Address
1	='9'	105
2	='23'	106

Example

Source Program

```
START 100
MOVER AREG, A
PRINT B
ADD BREG, ='9'
SUB BREG, D
COMP CREG, ='23'
LTORG

A DC 3
LABEL: EQU A
ORIGIN 500
L1: MULT CREG, ='7'
B DC 10
MOVEM CREG, ='7'
D DC 8
END
```

Target Code

```
    01 -  100
100)01 01 ---
101)09 -  ---
102)03 02 ---
103)04 02 ---
104)08 03 ---
105)-  -  009
106)-  -  023
107)-  -  003
No code
No code
500)05 03 ---
```

LC No	Incomplete Instruction
100	A
101	B
102	='9'
103	D
104	='23'
500	='7'

Sym_no	Symbol	Address
1	A	107
2	B	
3	D	
4	LABEL	107
5	L1	500

Lit_no	Literal	Address
1	='9'	105
2	='23'	106
3	='7'	

Example

Source Program	Target Code
START 100	01 - 100
MOVER AREG, A	100)01 01 ---
PRINT B	101)09 - ---
ADD BREG, ='9'	102)03 02 ---
SUB BREG, D	103)04 02 ---
COMP CREG, ='23'	104)08 03 ---
LTORG	105)- - 009
	106)- - 023
A DC 3	107)- - 003
LABEL: EQU A	No code
ORIGIN 500	No code
L1: MULT CREG, ='7'	500)05 03 ---
B DC 10	501)- - 010
MOVEM CREG, ='7'	
D DC 8	
END	

LC No	Incomplete Instruction
100	A
101	B
102	='9'
103	D
104	='23'
500	='7'

Sym_no	Symbol	Address
1	A	107
2	B	501
3	D	
4	LABEL	107
5	L1	500

Lit_no	Literal	Address
1	='9'	105
2	='23'	106
3	='7'	

Example

Source Program	Target Code
START 100	01 - 100
MOVER AREG, A	100)01 01 ---
PRINT B	101)09 - ---
ADD BREG, ='9'	102)03 02 ---
SUB BREG, D	103)04 02 ---
COMP CREG, ='23'	104)08 03 ---
LTORG	105)- - 009
	106)- - 023
A DC 3	107)- - 003
LABEL: EQU A	No code
ORIGIN 500	No code
L1: MULT CREG, ='7'	500)05 03 ---
B DC 10	501)- - 010
MOVEM CREG, ='7'	502)02 03 ---
D DC 8	
END	

LC No	Incomplete Instruction
100	A
101	B
102	='9'
103	D
104	='23'
500	='7'
502	='7'

Sym_no	Symbol	Address
1	A	107
2	B	501
3	D	
4	LABEL	107
5	L1	500

Lit_no	Literal	Address
1	='9'	105
2	='23'	106
3	='7'	

Example

Source Program / Target Code

```
START 100                    01 -  100
MOVER AREG, A            100)01 01 ---
PRINT B                  101)09 -  ---
ADD BREG, ='9'           102)03 02 ---
SUB BREG, D              103)04 02 ---
COMP CREG, ='23'         104)08 03 ---
LTORG                    105)-  -  009
                         106)-  -  023
A DC 3                   107)-  -  003
LABEL: EQU A             No code
ORIGIN 500               No code
L1: MULT CREG, ='7'      500)05 03 ---
B DC 10                  501)-  -  010
MOVEM CREG, ='7'         502)02 03 ---
D DC 8                   503)-  -  008
END
```

LC No	Incomplete Instruction
100	A
101	B
102	='9'
103	D
104	='23'
500	='7'
502	='7'

Sym_no	Symbol	Address
1	A	107
2	B	501
3	D	503
4	LABEL	107
5	L1	500

Lit_no	Literal	Address
1	='9'	105
2	='23'	106
3	='7'	

Example

Source Program	Target Code
START 100	01 - 100
MOVER AREG, A	100)01 01 ---
PRINT B	101)09 - ---
ADD BREG, ='9'	102)03 02 ---
SUB BREG, D	103)04 02 ---
COMP CREG, ='23'	104)08 03 ---
LTORG	105)- - 009
	106)- - 023
A DC 3	107)- - 003
LABEL: EQU A	No code
ORIGIN 500	No code
L1: MULT CREG, ='7'	500)05 03 ---
B DC 10	501)- - 010
MOVEM CREG, ='7'	502)02 03 ---
D DC 8	503)- - 008
END	504)- - 007

LC No	Incomplete Instruction
100	A
101	B
102	='9'
103	D
104	='23'
500	='7'
502	='7'

Sym_no	Symbol	Address
1	A	107
2	B	501
3	D	503
4	LABEL	107
5	L1	500

Lit_no	Literal	Address
1	='9'	105
2	='23'	106
3	='7'	504

Example

Source Program	Target Code
START 100	01 - 100
MOVER AREG, A	100)01 01 107
PRINT B	101)09 - ---
ADD BREG, ='9'	102)03 02 ---
SUB BREG, D	103)04 02 ---
COMP CREG, ='23'	104)08 03 ---
LTORG	105)- - 009
	106)- - 023
A DC 3	107)- - 003
LABEL: EQU A	No code
ORIGIN 500	No code
L1: MULT CREG, ='7'	500)05 03 ---
B DC 10	501)- - 010
MOVEM CREG, ='7'	502)02 03 ---
D DC 8	503)- - 008
END	504)- - 007

LC No	Incomplete Instruction
100	A
101	B
102	='9'
103	D
104	='23'
500	='7'
502	='7'

Sym_no	Symbol	Address
1	A	107
2	B	501
3	D	503
4	LABEL	107
5	L1	500

Lit_no	Literal	Address
1	='9'	105
2	='23'	106
3	='7'	504

Example

Source Program	Target Code
START 100	01 - 100
MOVER AREG, A	100)01 01 107
PRINT B	101)09 - 501
ADD BREG, ='9'	102)03 02 ---
SUB BREG, D	103)04 02 ---
COMP CREG, ='23'	104)08 03 ---
LTORG	105)- - 009
	106)- - 023
A DC 3	107)- - 003
LABEL: EQU A	No code
ORIGIN 500	No code
L1: MULT CREG, ='7'	500)05 03 ---
B DC 10	501)- - 010
MOVEM CREG, ='7'	502)02 03 ---
D DC 8	503)- - 008
END	504)- - 007

LC No	Incomplete Instruction
100	A
101	B
102	='9'
103	D
104	='23'
500	='7'
502	='7'

Sym_no	Symbol	Address
1	A	107
2	B	501
3	D	503
4	LABEL	107
5	L1	500

Lit_no	Literal	Address
1	='9'	105
2	='23'	106
3	='7'	504

Example

Source Program	Target Code
START 100	01 - 100
MOVER AREG, A	100)01 01 107
PRINT B	101)09 - 501
ADD BREG, ='9'	102)03 02 ---
SUB BREG, D	103)04 02 ---
COMP CREG, ='23'	104)08 03 ---
LTORG	105)- - 009
	106)- - 023
A DC 3	107)- - 003
LABEL: EQU A	No code
ORIGIN 500	No code
L1: MULT CREG, ='7'	500)05 03 ---
B DC 10	501)- - 010
MOVEM CREG, ='7'	502)02 03 ---
D DC 8	503)- - 008
END	504)- - 007

LC No	Incomplete Instruction
100	A
101	B
102	='9'
103	D
104	='23'
500	='7'
502	='7'

Sym_no	Symbol	Address
1	A	107
2	B	501
3	D	503
4	LABEL	107
5	L1	500

Lit_no	Literal	Address
1	='9'	105
2	='23'	106
3	='7'	504

Example

Source Program

```
START 100
MOVER AREG, A
PRINT B
ADD BREG, ='9'
SUB BREG, D
COMP CREG, ='23'
LTORG

A DC 3
LABEL: EQU A
ORIGIN 500
L1: MULT CREG, ='7'
B DC 10
MOVEM CREG, ='7'
D DC 8
END
```

Target Code

```
    01 -  100
100)01 01 107
101)09 -  501
102)03 02 105
103)04 02 ---
104)08 03 ---
105)-  -  009
106)-  -  023
107)-  -  003
No code
No code
500)05 03 ---
501)-  -  010
502)02 03 ---
503)-  -  008
504)-  -  007
```

LC No	Incomplete Instruction
100	A
101	B
102	='9'
103	D
104	='23'
500	='7'
502	='7'

Sym_no	Symbol	Address
1	A	107
2	B	501
3	D	503
4	LABEL	107
5	L1	500

Lit_no	Literal	Address
1	='9'	105
2	='23'	106
3	='7'	504

Example

Source Program / Target Code

```
START 100                     01 -  100
MOVER AREG, A             100)01 01 107
PRINT B                   101)09 -  501
ADD BREG, ='9'            102)03 02 105
SUB BREG, D               103)04 02 503
COMP CREG, ='23'          104)08 03 ---
LTORG                     105)-  -  009
                          106)-  -  023
A DC 3                    107)-  -  003
LABEL: EQU A              No code
ORIGIN 500                No code
L1: MULT CREG, ='7'       500)05 03 ---
B DC 10                   501)-  -  010
MOVEM CREG, ='7'          502)02 03 ---
D DC 8                    503)-  -  008
END                       504)-  -  007
```

LC No	Incomplete Instruction
100	A
101	B
102	='9'
103	D
104	='23'
500	='7'
502	='7'

Sym_no	Symbol	Address
1	A	107
2	B	501
3	D	503
4	LABEL	107
5	L1	500

Lit_no	Literal	Address
1	='9'	105
2	='23'	106
3	='7'	504

Example

Source Program / **Target Code**

```
START 100                    01 -  100
MOVER AREG, A           100)01 01 107
PRINT B                 101)09 -  501
ADD BREG, ='9'          102)03 02 105
SUB BREG, D             103)04 02 503
COMP CREG, ='23'        104)08 03 106
LTORG                   105)-  -  009
                        106)-  -  023
A DC 3                  107)-  -  003
LABEL: EQU A            No code
ORIGIN 500              No code
L1: MULT CREG, ='7'     500)05 03 ---
B DC 10                 501)-  -  010
MOVEM CREG, ='7'        502)02 03 ---
D DC 8                  503)-  -  008
END                     504)-  -  007
```

LC No	Incomplete Instruction
100	A
101	B
102	='9'
103	D
104	='23'
500	='7'
502	='7'

Sym_no	Symbol	Address
1	A	107
2	B	501
3	D	503
4	LABEL	107
5	L1	500

Lit_no	Literal	Address
1	='9'	105
2	='23'	106
3	='7'	504

Example

Source Program	Target Code
START 100	01 - 100
MOVER AREG, A	100)01 01 107
PRINT B	101)09 - 501
ADD BREG, ='9'	102)03 02 105
SUB BREG, D	103)04 02 503
COMP CREG, ='23'	104)08 03 106
LTORG	105)- - 009
	106)- - 023
A DC 3	107)- - 003
LABEL: EQU A	No code
ORIGIN 500	No code
L1: MULT CREG, ='7'	500)05 03 504
B DC 10	501)- - 010
MOVEM CREG, ='7'	502)02 03 ---
D DC 8	503)- - 008
END	504)- - 007

LC No	Incomplete Instruction
100	A
101	B
102	='9'
103	D
104	='23'
500	='7'
502	='7'

Sym_no	Symbol	Address
1	A	107
2	B	501
3	D	503
4	LABEL	107
5	L1	500

Lit_no	Literal	Address
1	='9'	105
2	='23'	106
3	='7'	504

Example

Source Program	Target Code
START 100	01 - 100
MOVER AREG, A	100)01 01 107
PRINT B	101)09 - 501
ADD BREG, ='9'	102)03 02 105
SUB BREG, D	103)04 02 503
COMP CREG, ='23'	104)08 03 106
LTORG	105)- - 009
	106)- - 023
A DC 3	107)- - 003
LABEL: EQU A	No code
ORIGIN 500	No code
L1: MULT CREG, ='7'	500)05 03 504
B DC 10	501)- - 010
MOVEM CREG, ='7'	502)02 03 504
D DC 8	503)- - 008
END	504)- - 007

LC No	Incomplete Instruction
100	A
101	B
102	='9'
103	D
104	='23'
500	='7'
502	='7'

Sym_no	Symbol	Address
1	A	107
2	B	501
3	D	503
4	LABEL	107
5	L1	500

Lit_no	Literal	Address
1	='9'	105
2	='23'	106
3	='7'	504

Error Handling in Assemblers

Why Error Handling is Important

- Ensures correctness of generated object code
- Prevents invalid machine instructions
- Improves debugging experience
- Detects logical and syntactic mistakes early
- Essential for reliable system software

An assembler must detect errors without producing incorrect executable code.

Types of Errors in Assemblers

- Lexical Errors
- Syntax Errors
- Semantic Errors
- Symbol-Related Errors
- Addressing / Range Errors
- Assembler Directive Errors

Types of Errors in Assemblers

- Lexical Errors
- Syntax Errors
- Semantic Errors
- Symbol-Related Errors
- Addressing / Range Errors
- Assembler Directive Errors

Errors

Lexical Errors

- Invalid characters in symbol name
- Illegal literal format
- Missing quotes in string literal

MOV AREG, =5'

Syntax Errors

Violation of assembly language grammar.

- Missing operand
- Wrong instruction format

ADD AREG

Errors

Semantic Errors

Program is syntactically correct but logically invalid.

- Invalid register name
- Operand type mismatch
- Using memory operand where register required

ADD 5, AREG

Symbol-Related Errors

- Duplication of symbol
- Undefined Symbol
- Unresolved Forward reference

ADD AREG, X

Errors

Addressing and Range Errors

- Address out of machine limit
- Instruction size overflow
- Memory overlap
- Branch target out of range

`JMP 50000`

Directive-Related Errors

- Missing START or END
- Multiple START statements
- Invalid ORIGIN expression

`ORIGIN A+5` (if A undefined)

Error Detection in Two-Pass Assembler

Pass I detects:

- Duplicate symbols
- Invalid opcodes
- Incorrect directives

Pass II detects:

- Undefined symbols
- Invalid addresses
- Encoding errors

Error Detection in One-Pass Assembler

More complex because:

- Code is generated immediately
- Forward references must be tracked
- Some errors discovered late

If symbol remains undefined at END:

- Report error
- Do not finalize object code